

PROGRAMACIÓN AVANZADA

Sesión 23

Argumentos de consola

→ [NOTAS DE CLASE](#)

Índice

1	1969: “haz una sola cosa y hazla bien”	2
2	main con parámetros: argc y argv	2
	2.1 Un primer programa con argc/argv	3
3	Verificar el número de argumentos	4
4	Convertir argumentos a números	5
5	Actividad: el teclado numérico	6
6	Ejercicios	8

1969: “haz una sola cosa y hazla bien”

En 1969, Ken Thompson diseñó la primera versión de Unix en un PDP-7 que nadie más usaba en los Laboratorios Bell. Dennis Ritchie se unió poco después, y juntos establecieron una filosofía de diseño que sigue siendo vigente más de cincuenta años después:

“Escribe programas que hagan una sola cosa y la hagan bien.”
“Escribe programas que puedan trabajar juntos.”

Para que los programas trabajen juntos necesitan dos mecanismos: una forma de recibir instrucciones desde el exterior, y una forma de pasar sus resultados a otro programa. La segunda se resolvió con las **tuberías** (*pipes*); la primera, con los **argumentos de consola**.

Cuando escribes:

```
$ g++ -O2 -o hola hola.cpp
```

el shell toma esa línea, la separa en cinco cadenas y se las pasa al compilador. El compilador las recibe como un arreglo de cadenas llamado `argv` (de *argument vector*) y un entero llamado `argc` (de *argument count*). Llevas un semestre usando `g++` sin saber que le estás pasando cuatro argumentos por `argv`.

Esta convención fue diseñada por Dennis Ritchie al desarrollar el lenguaje C alrededor de 1972 y quedó formalizada en el estándar ANSI C de 1989. Hoy es parte del estándar de C++ y de prácticamente todo lenguaje compilado moderno.

Los programas `grep`, `wc`, `sort` y `ls` son extremadamente pequeños porque cada uno hace una sola cosa. Su poder real proviene de combinarlos:

```
$ cat notas.txt | grep "error" | wc -l
```

Este comando cuenta cuántas líneas de `notas.txt` contienen la palabra “error”. Cada programa recibe su configuración por `argv` y sus datos por la entrada estándar.

Referencia: Ritchie, D. M., & Thompson, K. (1974). “The UNIX time-sharing system.” *Communications of the ACM*, 17(7), 365–375.

main con parámetros: argc y argv

Hasta ahora siempre has escrito:

```
int main(){
    ...
}
```

Esa forma no recibe argumentos. La versión completa de `main` es:

```
int main(int argc, char* argv[]){  
    ...  
}
```

argc Número de cadenas recibidas, incluyendo el nombre del programa. Su valor mínimo es 1.

argv Arreglo de cadenas estilo C. Cada elemento es un `char*` que apunta a una cadena terminada en `'\0'`.

- `argv[0]` : nombre o ruta del ejecutable.
- `argv[1]` a `argv[argc-1]` : los argumentos del usuario.

Ejemplo: si ejecutas:

```
$ ./mi_programa hola mundo 42
```

entonces:

Variable	Valor	Descripción
<code>argc</code>	4	cuatro cadenas en total
<code>argv[0]</code>	<code>"./mi_programa"</code>	nombre del ejecutable
<code>argv[1]</code>	<code>"hola"</code>	primer argumento
<code>argv[2]</code>	<code>"mundo"</code>	segundo argumento
<code>argv[3]</code>	<code>"42"</code>	tercer argumento

`argv[3]` es la cadena `"42"`, no el número 42. Si quieres usarlo en una operación aritmética hay que convertirlo explícitamente. Este es uno de los errores más comunes al trabajar con argumentos de consola.

• Un primer programa con `argc/argv`

hola_args.cpp:

```
#include <iostream>  
using namespace std;  
  
int main(int argc, char* argv[]){  
    cout << "argc = " << argc << endl;  
    for (int i = 0; i < argc; i++){  
        cout << "argv[" << i << "] = " << argv[i] << endl;  
    }  
}
```

Ejecución:

```
$ ./hola_args uno dos tres
argc = 4
argv[0] = ./hola_args
argv[1] = uno
argv[2] = dos
argv[3] = tres
```

✓ Recuerda

El shell divide la línea de comandos usando los espacios como separadores. Si quieres pasar una cadena con espacios como un solo argumento, enciérrala en comillas:

```
$ ./programa "hola mundo"
```

En ese caso `argc` es 2 y `argv[1]` es "hola mundo" (una sola cadena, no dos).

Verificar el número de argumentos

Los programas robustos siempre verifican que recibieron el número correcto de argumentos antes de intentar usarlos. Acceder a `argv[2]` cuando `argc` es 2 es comportamiento indefinido; en la práctica provoca un *segmentation fault* o lee memoria basura.

uso.cpp:

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]){
    if (argc != 3){
        cerr << "Uso: " << argv[0]
            << " <nombre> <apellido>" << endl;
        return 1;
    }
    cout << "Hola, " << argv[1] << " " << argv[2] << "!" << endl;
}
```

Ejecuciones:

```
$ ./uso Ana Lopez
Hola, Ana Lopez!
```

```
$ ./uso Ana
Uso: ./uso <nombre> <apellido>
```

Dos detalles importantes:

1. El mensaje de error va a `cerr` (la salida de error estándar), no a `cout`. Así el usuario puede redirigir la salida normal sin mezclarla con mensajes de error.

2. El programa devuelve `1` para indicar al shell que algo salió mal. Por convención, `return 0` significa éxito y cualquier otro valor indica error.

Usa `argv[0]` en el mensaje de uso en lugar de escribir el nombre del programa a mano. Así el mensaje sigue siendo correcto aunque renombres el ejecutable.

Convertir argumentos a números

Los argumentos siempre llegan como cadenas de texto. Para usarlos en operaciones numéricas hay que convertirlos. C++ ofrece varias funciones para esto:

Función	Tipo que devuelve	Encabezado
<code>atoi(s)</code>	<code>int</code>	<code><cstdlib></code>
<code>atol(s)</code>	<code>long</code>	<code><cstdlib></code>
<code>atof(s)</code>	<code>double</code>	<code><cstdlib></code>
<code>stoi(s)</code>	<code>int</code>	<code><string></code>
<code>stol(s)</code>	<code>long</code>	<code><string></code>
<code>stod(s)</code>	<code>double</code>	<code><string></code>

Las funciones que comienzan con *a* (de *ASCII to*) vienen de C y reciben `char*`. Las que comienzan con *s* (de *string to*) son de C++ y aceptan `std::string`. Como `argv[i]` ya es `char*`, las versiones de C funcionan directamente sin conversión extra.

suma.cpp — suma todos los argumentos como enteros:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char* argv[]){
    int total = 0;
    for (int i = 1; i < argc; i++){
        total += atoi(argv[i]);
    }
    cout << total << endl;
}
```

Ejecución:

```
$ ./suma 10 20 30 5
65
```

`atoi` devuelve `0` cuando la cadena no es un número válido, sin ningún aviso. `stoi` lanza una excepción en ese caso. Si los argumentos vienen directamente del usuario y pueden ser inválidos, `stoi` dentro de un bloque `try / catch` es la

opción más segura.

calc.cpp:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char* argv[]){
    if (argc != 4){
        cerr << "Uso: " << argv[0]
            << " <num1> <op> <num2>" << endl;
        return 1;
    }
    double a = atof(argv[1]);
    char op = argv[2][0];
    double b = atof(argv[3]);

    if (op == '+') cout << a + b << endl;
    else if (op == '-') cout << a - b << endl;
    else if (op == '*') cout << a * b << endl;
    else if (op == '/') cout << a / b << endl;
    else {
        cerr << "Operador desconocido: " << op << endl;
        return 1;
    }
}
```

Ejecuciones:

```
$ ./calc 3.5 + 1.5
5
$ ./calc 10 / 4
2.5
```

Actividad: el teclado numérico

Antes de los *smartphones*, todos los teléfonos celulares tenían un teclado de doce teclas. Para escribir un mensaje de texto había que presionar cada tecla varias veces: el número de pulsaciones indicaba qué letra se quería.

El detalle importante es que la **primera pulsación de una tecla siempre introduce el dígito**, no una letra. Las letras se obtienen a partir de la segunda pulsación. Con el mapeo estándar (ITU E.161, 1994):

Tecla	Letras	Pulsaciones para cada letra			
2	a b c	22	222	2222	
3	d e f	33	333	3333	
4	g h i	44	444	4444	
5	j k l	55	555	5555	
6	m n o	66	666	6666	
7	p q r s	77	777	7777	77777
8	t u v	88	888	8888	
9	w x y z	99	999	9999	99999

La regla general: para la n -ésima letra de una tecla se presiona ese dígito exactamente $n + 1$ veces (una por el dígito más n por la posición). Los espacios entre palabras se representan con un espacio en la salida.

• Ejemplo verificado

```
$ ./convertir hola luna
4446666555522 555588866622
```

Comprobación de hola:

Letra	Tecla	Posición	Pulsaciones	Salida
h	4 (ghi)	2 ^a	$2 + 1 = 3$	444
o	6 (mno)	3 ^a	$3 + 1 = 4$	6666
l	5 (jkl)	3 ^a	$3 + 1 = 4$	5555
a	2 (abc)	1 ^a	$1 + 1 = 2$	22

hola $\rightarrow 444 + 6666 + 5555 + 22 = 4446666555522$

Comprobación de luna:

Letra	Tecla	Posición	Pulsaciones	Salida
l	5 (jkl)	3 ^a	$3 + 1 = 4$	5555
u	8 (tuv)	2 ^a	$2 + 1 = 3$	888
n	6 (mno)	2 ^a	$2 + 1 = 3$	666
a	2 (abc)	1 ^a	$1 + 1 = 2$	22

luna $\rightarrow 5555 + 888 + 666 + 22 = 555588866622$

• Especificación del programa

- El programa se llama `convertir` y recibe las palabras como argumentos de consola (cada palabra es un `argv[i]`).
- Cada argumento contiene solo letras minúsculas del alfabeto inglés.
- La salida imprime las pulsaciones de cada palabra separadas por un espacio, seguidas de un salto de línea.
- Para simplificar, ningún argumento contendrá dos letras consecutivas en la misma tecla (por ejemplo, no habrá "gh" ni "mn" juntas dentro de la misma palabra).

• Pista

Observa que las letras del alfabeto inglés van de `'a'` (ASCII 97) a `'z'` (ASCII 122). La expresión `letra - 'a'` produce un índice de 0 a 25 que puedes usar para buscar en una tabla. Piensa en qué datos necesitas guardar por cada letra: su tecla y su posición en esa tecla.

Ejercicios

1. Escribe un programa **eco.cpp** que imprima cada argumento en una línea separada, numerado desde 1. Ejemplo:

```
$ ./eco rojo verde azul
1: rojo
2: verde
3: azul
```

Si no se proporcionan argumentos, el programa no imprime nada (sin mensaje de error).

2. Escribe un programa **mayor.cpp** que reciba varios enteros como argumentos e imprima el mayor de ellos. Si no se proporciona ningún argumento, muestra un mensaje de uso en `cerr` y termina con código 1.
3. Escribe un programa **reversa.cpp** que imprima los argumentos en orden inverso (el último primero, el primero al final). Ejemplo:

```
$ ./reversa uno dos tres
tres
dos
uno
```

4. Escribe un programa **contador.cpp** que reciba como primer argumento una letra y como argumentos siguientes varias palabras, y cuente cuántas veces aparece esa letra en el conjunto de palabras. Ejemplo:

```
$ ./contador a banana manzana naranja
7
```

5. Escribe el programa **convertir.cpp** de la actividad. Antes de entregarlo, incluye al inicio del archivo, como comentarios, los cálculos manuales de al menos dos palabras de prueba distintas que usaste para verificar tu solución.

Resumen

- `main` puede recibir dos parámetros: `int argc` (número de cadenas, incluyendo el nombre del programa) y `char* argv[]` (arreglo de cadenas C con cada argumento).
- `argv[0]` siempre es el nombre del ejecutable; los argumentos del usuario están en `argv[1]` a `argv[argc-1]`.
- Siempre verifica el valor de `argc` antes de acceder a `argv[i]`. Leer fuera de rango es comportamiento indefinido.
- Los argumentos llegan como cadenas; `atoi`, `atof` y `stoi` permiten convertirlos a enteros o flotantes.
- El mensaje de uso convencional se imprime en `cerr` y el programa retorna 1 para señalar error al shell.
- En el teclado numérico telefónico, la n -ésima letra de una tecla requiere $n + 1$ pulsaciones: una por el dígito y n por su posición en la tecla.
- La filosofía Unix de “hacer una cosa y hacerla bien” se apoya en los argumentos de consola: cada programa recibe su configuración por `argv` y puede encadenarse con otros mediante tuberías.